ChatGPT

Analisi del sistema di lock in *util.js*

Nel codice attuale di `util.js` il sistema di lock di una prenotazione (`state.context.lock`) presenta vari difetti logici che lo rendono debole e incoerente. In particolare:

- **Merge errato in** `bumpBookingLock` **e** `unlockBooking`: viene usato lo spread operator (`...prev`) in ordine sbagliato, così campi appena impostati (come `active`, `ts`, `ttl`) vengono subito sovrascritti dai valori precedenti. Per esempio in `bumpBookingLock` si fa prima `{ active: true, ts: now, ttl, ...prev, ...meta }`: il successivo `...prev` può riportare a `active: false` o valori vecchi di `ts`, vanificando il lock (in JavaScript, quando più oggetti hanno la stessa proprietà in un merge, prevale l'ultima definizione [1]). Lo stesso succede in `unlockBooking`.

- **Formati di lock incoerenti**: il codice usa due schemi diversi per il lock. La funzione `lockBooking(state)` salva `{ type: "appointment", until: timestamp }`, mentre gli altri metodi (`isBookingLocked`, `bumpBookingLock`, `unlockBooking`) usano `{ active: bool, ts: number, ttl: number }`. Se lo stato di lock viene salvato con `{type, until}` ma poi controllato come `{active, ts, ttl}`, il lock sarà interpretato erroneamente. Ad esempio, `isBookingLocked` verifica `lock.active`, ma nel formato `{type,until}` non esiste `active`, quindi il lock non viene riconosciuto o rimane "acceso" anche se dovrebbe essere scaduto.

- **Mancanza di validazioni e conflitti nei metadati**: non si filtrano campi di metadati con lo stesso nome di quelli core (`active`, `ts`, `ttl`), permettendo a un eventuale `meta` di override indesiderati. Inoltre, funzioni come `unlockBooking` non usano `ensureContext(st)`, perciò se `st.context` non esiste il metodo esce silenziosamente senza inizializzare correttamente lo stato.

Problemi specifici con esempi di codice

1. `bumpBookingLock`: merge inverso dei campi core

Codice attuale (bug):

```
function bumpBookingLock(st, ttlMs = 2 * 60 * 1000, meta) {
  if (!st) return;
  ensureContext(st);
  const rawTtl = Number(ttlMs);
  const ttl = Number.isFinite(rawTtl) && rawTtl > 0 ? rawTtl : 2 * 60 * 1000;
  const now = Date.now();
  const prev = st.context.lock || {};

  // *Errore*: i campi core sono scritti prima, poi li sovrascrive ...prev
  st.context.lock = {
    active: true,
```

```
    ts: now,
    ttl,
    ...prev,
    ...(meta && typeof meta === "object" ? meta : null),
  };
}
```

Qui, se in `prev` c'è ad esempio `active: false`, lo spread `...prev` lo riporta e il lock rimane spento. In JavaScript l'ultima proprietà definita prevale [1], quindi l'ordine degli spread è cruciale.

2. `unlockBooking`: merge che annulla lo sblocco

Codice attuale (bug):

```
function unlockBooking(st, reason) {
  if (!st || !st.context) return;
  const now = Date.now();
  const prev = st.context.lock || {};

  // *Errore*: attivo=false viene sovrascritto da ...prev se prev.active=true
  st.context.lock = {
    active: false,
    ts: 0,
    ttl: 0,
    ...prev,
    releasedAt: now,
    releaseReason: reason || "manual",
  };
}
```

Analogamente, se `prev.active` era `true`, il lock tornerà erroneamente attivo perché `...prev` appare dopo. In pratica il primo blocco può essere annullato dal secondo.

3. Formati di lock differenti nel contesto

Il metodo `lockBooking(state)` attuale fa:

```
function lockBooking(state) {
  state.context.lock = { type: "appointment", until: now() +
BOOKING_LOCK_MS };
}
```

Salva **solo** `{type, until}` e nessuno dei campi `active/ts/ttl`. Invece `isBookingLocked(state)` controlla esclusivamente `lock.active`, `lock.ts` e `lock.ttl` per determinare se il lock è valido. Esempio:

```
function isBookingLocked(st, nowTs) {
  if (!st || !st.context || !st.context.lock) return false;
  const lock = st.context.lock;
  if (!lock.active) return false; // NB: non c'è active in formato
{type,until}
  // ...controlli di ts/ttl...
}
```

Confrontando i due schemi:

- **Formato 1** (usato in *lockBooking*):
  `{ type: "appointment", until: <timestamp> }`
- **Formato 2** (usato altrove):
  `{ active: <bool>, ts: <timestamp>, ttl: <millisecondi>, ... }`

Se si salva un lock con `{type, until}`, `isBookingLocked` vedrà `lock.active === undefined` (falsario) e ritornerà subito `false`, ignorando completamente il lock. In alternativa, non si gestisce quando `type,until` scadono. Questo provoca incoerenza: il lock potrebbe rimanere attivo "per sempre" o venire ignorato.

Soluzioni e refactoring consigliati

Per risolvere i problemi dobbiamo unificare il formato del lock e correggere l'ordine dei merge. In sintesi:

- **Unificare il formato del lock**: adottare un unico schema dati per il lock, ad esempio `{ active: boolean, ts: number, ttl: number, type?: string, … }`. Si elimina quindi il formato `{type,until}` e si converte `lockBooking` per usare `active/ts/ttl`.
- **Correggere l'ordine di spread**: quando si crea/aggiorna `st.context.lock`, fondiamo prima i metadati esistenti e poi sovrascriviamo i campi core (`active`, `ts`, `ttl`) **per ultimi**. In questo modo la nuova impostazione prevale sempre sulla vecchia [1] [2].
- **Filtrare metadati riservati**: se si accetta un oggetto `meta`, escludere da esso chiavi come `active`, `ts`, `ttl` in modo che non possano accidentalmente sovrascrivere valori importanti.
- **Usare** `ensureContext` **sempre**: chiamarlo in tutte le funzioni che manipolano `state.context.lock` (anche `unlockBooking` e `lockBooking`) per garantire che `st.context` e `st.context.lock` siano sempre inizializzati.
- **Validazione**: mantenere i controlli già presenti in `isBookingLocked` (es. `Number.isFinite(ts)`) e gestire esplicitamente lock corrotti o scaduti, azzerando lo stato e magari aggiungendo log (come fa già con `lastExpiredAt`).

Esempio di codice ristrutturato

Prima (bug in `bumpBookingLock`):

```
function bumpBookingLock(st, ttlMs, meta) {
  ensureContext(st);
  const now = Date.now();
  const prev = st.context.lock || {};
  st.context.lock = {
    active: true,
    ts: now,
    ttl: ttlMs,
    ...prev,  // ERRORE: prev sovrascrive active/ts/ttl
    ...meta,
  };
}
```

Dopo (merge corretto):

```
function bumpBookingLock(st, ttlMs = 2 * 60 * 1000, meta) {
  ensureContext(st);
  const rawTtl = Number(ttlMs);
  const ttl = Number.isFinite(rawTtl) && rawTtl > 0 ? rawTtl : 2 * 60 * 1000;
  const now = Date.now();

  // Preparo il nuovo lock fondendo prev+meta e sovrascrivendo con campi core
  const prev = st.context.lock || {};

  // Rimuovo chiavi riservate da meta per sicurezza
  const { active, ts, ttl: _ttl, ...metaFiltered } = meta || {};

  // Creazione sicura dell'oggetto lock
  st.context.lock = {
    ...prev,          // mantengo vecchi metadati
    active: true,
    ts: now,
    ttl: ttl,
    ...metaFiltered   // aggiungo eventuali nuovi metadati (senza campi
riservati)
  };
}
```

In questo modo, anche se `prev` conteneva `active` o `ts`, verrà sovrascritto da quelli nuovi. L'ultima assegnazione (qui di fatto `active: true`, `ts: now`, `ttl: ttl`) prevale [1].

Lo stesso principio vale per `unlockBooking`:

Prima (bug):

```
function unlockBooking(st, reason) {
  if (!st || !st.context) return;
```

```
  const now = Date.now();
  const prev = st.context.lock || {};
  st.context.lock = {
    active: false,
    ts: 0,
    ttl: 0,
    ...prev,  // ERRORE: ripristina active/ts/ttl precedenti
    releasedAt: now,
    releaseReason: reason || "manual",
  };
}
```

Dopo (merge corretto):

```
function unlockBooking(st, reason) {
  if (!st) return;
  ensureContext(st);
  const now = Date.now();
  const prev = st.context.lock || {};

  // Rimuovo da prev eventuali campi core (opzionale, già sovrascritti sotto)
  const { active, ts, ttl, ...prevFiltered } = prev;

  st.context.lock = {
    ...prevFiltered,  // mantengo vecchi metadati (owner, reason, ecc.)
    active: false,
    ts: 0,
    ttl: 0,
    releasedAt: now,
    releaseReason: reason || "manual",
  };
}
```

Qui `active: false` viene assegnato dopo i metadati, quindi eventuali `active` nel vecchio stato vengono sovrascritti correttamente. Abbiamo anche spostato `ensureContext` in apertura per simmetria.

Refactor di `lockBooking` e `isBookingLocked`

Per coerenza, modifichiamo `lockBooking(state)` per usare lo stesso formato. Ad esempio:

```
function lockBooking(state) {
  if (!state) return;
  ensureContext(state);
  const now = Date.now();
  state.context.lock = {
    active: true,
    ts: now,
```

```
    ttl: BOOKING_LOCK_MS,    // durata del lock
    type: "appointment",     // metadato opzionale
  };
}
```

Così `isBookingLocked` (che controlla `active/ts/ttl`) funzionerà correttamente. In alternativa si può delegare a `bumpBookingLock`:

```
function lockBooking(state) {
  bumpBookingLock(state, BOOKING_LOCK_MS, { type: "appointment" });
}
```

Inoltre, potremmo aggiornare `isBookingLocked` per gestire eventuali vecchi lock `{type,until}` (per retrocompatibilità), convertendoli automaticamente al formato nuovo. Per esempio:

```
function isBookingLocked(st, nowTs) {
  if (!st || !st.context || !st.context.lock) return false;
  let lock = st.context.lock;

  // Se riconosciamo il vecchio formato {type, until}, lo converto
  if (lock.type && typeof lock.until === "number") {
    const now = typeof nowTs === "number" ? nowTs : Date.now();
    if (now < lock.until) {
      // Imposto un nuovo lock attivo con TTL residuo
      const remaining = lock.until - now;
      st.context.lock = { active: true, ts: now, ttl: remaining, type:
lock.type };
      lock = st.context.lock;
    } else {
      // Lock scaduto
      st.context.lock = { active: false, ts: 0, ttl: 0 };
      return false;
    }
  }

  // Procedo con la logica originale su active/ts/ttl
  if (!lock.active) return false;
  const now = typeof nowTs === "number" ? nowTs : Date.now();
  const ts = Number(lock.ts);
  const ttl = Number.isFinite(lock.ttl) && lock.ttl > 0 ? lock.ttl : (2 * 60
* 1000);

  if (!Number.isFinite(ts) || ts <= 0) {
    st.context.lock = { active: false, ts: 0, ttl: 0 };
    return false;
  }
  if (now < ts || now - ts > ttl) {
    st.context.lock = { active: false, ts: 0, ttl: 0, lastExpiredAt: now,
lastExpireTtl: ttl };
```

```
    return false;
  }
  return true;
}
```

Così evitiamo comportamento indefinito se troviamo ancora vecchi lock. Tuttavia, la soluzione migliore è eliminare definitivamente il doppio formato e usare sempre `{active, ts, ttl}` come unico standard.

Validazioni aggiuntive e pratiche sicure

- **Validazione dei parametri**: Convertire `ttlMs` in numero e usare `Number.isFinite` come fatto nel codice per evitare valori non numerici o negativi. Allo stesso modo assicurarsi che `state` e `state.context` esistano (con `ensureContext`) prima di modificare il lock.
- **Atomicità**: Creare sempre un nuovo oggetto (`st.context.lock = { ... }`) anziché mutare parzialmente un oggetto esistente, per evitare stati intermedi non validi.
- **Lock pulito sui guasti**: In `isBookingLocked` il comportamento di reset automatico (setto `active: false` se dati corrotti o lock scaduto) è utile per evitare che un lock invalidi permanentemente la prenotazione. È buona pratica includere anche log o metadati (es. `lastExpiredAt`) per il debug.
- **Separazione dei metadati**: Tenere distinti i campi di stato (`active, ts, ttl`) dai metadati (ad esempio `owner`, `reason`, `type`). Nel refactor sopra si è mostrato come estrarre e filtrare i metadati in modo da non interferire con lo stato.
- **Documentazione e test**: Aggiornare i commenti in codice e scrivere test automatici per i casi di lock. Ad esempio, testare che `bumpBookingLock` con merge inverso blocchi correttamente `active=true` indipendentemente da `prev`.

Esempio riepilogativo di codice corretto

```
// Inizializza lo stato se manca, assicura formato base
function ensureContext(st) {
  if (!st) return;
  st.context = st.context || {};
  st.context.lock = st.context.lock || { active: false, ts: 0, ttl: 0 };
}

// Ritorna true se c'è un lock valido
function isBookingLocked(st, nowTs) {
  if (!st || !st.context || !st.context.lock) return false;
  let lock = st.context.lock;

  // (Opzionale) Converti formato legacy {type, until}
  if (lock.type && typeof lock.until === "number") {
    const now = typeof nowTs === "number" ? nowTs : Date.now();
    if (now < lock.until) {
```

```
      st.context.lock = { active: true, ts: now, ttl: lock.until - now,
type: lock.type };
      lock = st.context.lock;
    } else {
      st.context.lock = { active: false, ts: 0, ttl: 0 };
      return false;
    }
  }

  if (!lock.active) return false;
  const now = typeof nowTs === "number" ? nowTs : Date.now();
  const ts = Number(lock.ts);
  const ttl = Number.isFinite(lock.ttl) && lock.ttl > 0 ? lock.ttl : (2 * 60
* 1000);
  if (!Number.isFinite(ts) || ts <= 0) {
    st.context.lock = { active: false, ts: 0, ttl: 0 };
    return false;
  }
  if (now < ts || now - ts > ttl) {
    st.context.lock = { active: false, ts: 0, ttl: 0, lastExpiredAt: now,
lastExpireTtl: ttl };
    return false;
  }
  return true;
}

// Aggiorna/crea il lock: campi core per ultimi
function bumpBookingLock(st, ttlMs = 2 * 60 * 1000, meta) {
  if (!st) return;
  ensureContext(st);
  const rawTtl = Number(ttlMs);
  const ttl = Number.isFinite(rawTtl) && rawTtl > 0 ? rawTtl : 2 * 60 * 1000;
  const now = Date.now();
  const prev = st.context.lock || {};

  // Escludo chiavi riservate dai metadati
  const { active, ts, ttl: ___, ...prevMeta } = prev;
  const { active: _, ts: __, ttl: ___2, ...metaFiltered } = meta || {};

  st.context.lock = {
    ...prevMeta,       // metadati precedenti (owner, reason, type, ecc.)
    active: true,
    ts: now,
    ttl: ttl,
    ...metaFiltered    // nuovi metadati (es. owner, type)
  };
}

// Sblocca la prenotazione: attiva=false per ultimi
function unlockBooking(st, reason) {
  if (!st) return;
```

```
  ensureContext(st);
  const now = Date.now();
  const prev = st.context.lock || {};

  const { active, ts, ttl, ...prevMeta } = prev;
  st.context.lock = {
    ...prevMeta,
    active: false,
    ts: 0,
    ttl: 0,
    releasedAt: now,
    releaseReason: reason || "manual",
  };
}

// Esempio di lockBooking che usa lo stesso formato
function lockBooking(state) {
  if (!state) return;
  ensureContext(state);
  const now = Date.now();
  state.context.lock = {
    active: true,
    ts: now,
    ttl: BOOKING_LOCK_MS,
    type: "appointment"
  };
}
```

Best practices per il futuro

- **Formato dati unico e documentato**: definire uno schema chiaro per il lock (ad es. tramite JSDoc o TypeScript). Ciò impedisce di mescolare formati diversi e facilita l'estendibilità.
- **Immutabilità locale**: ogni funzione di aggiornamento `st.context.lock = {...}` crea un nuovo oggetto. Questo evita effetti collaterali imprevisti se altri riferimenti condividono lo stesso oggetto.
- **Chiamare** `ensureContext` **ovunque**: prima di leggere o scrivere `state.context.lock`, assicurarsi che `state` e `state.context` esistano (come già fa `bumpBookingLock`).
- **Gestione dei metadati**: evitare che oggetti esterni possano inserire proprietà riservate (filtrare o validare `meta`); in generale, separare i campi di stato dai metadati opzionali.
- **Testing e log**: implementare test per i casi limite (lock scaduto, tempo che torna indietro, dati corrotti) e mantenere tracciatura di errori nei campi di debug (`lastExpiredAt`, `releaseReason`).

Con queste modifiche il sistema di lock sarà molto più robusto: l'ordine di fusione degli oggetti sarà corretto (come descritto dalla documentazione JavaScript sulle proprietà duplicate [1] [2]), il formato sarà coerente, e le funzioni di lock/unlock opereranno in modo prevedibile e sicuro.

Fonti: l'uso dello spread operator in JavaScript copia proprietà in ordine [3] [1]; le proprietà con chiavi duplicate vengono sovrascritte da quelle nell'ultima fonte [2] [1], quindi l'ordine di fusione è critico. Questi principi guidano il refactoring sopra illustrato.

1 3 Spread syntax (...) - JavaScript | MDN
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Spread_syntax

2 Object.assign() - JavaScript | MDN
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/assign